

APOSTILA COMPLETA: Arquitetura de Software para E-commerce Guia Definitivo do Desenvolvedor Profissional

ÍNDICE Introdução

Fundamentos do Desenvolvimento Profissional

Arquitetura de Software

Estrutura de Projetos

Padrões de Design

Testes e Qualidade

DevOps e CI/CD

Documentação

Passo a Passo Prático

Checklists e Templates

1. INTRODUÇÃO {#introducao} 1.1 Sobre esta Apostila Esta apostila foi criada para ser seu guia definitivo para desenvolver aplicações profissionais, utilizando as melhores práticas do mercado, independentemente da linguagem de programação.

1.2 Objetivos ☒ Ensinar arquitetura de software do zero

☒ Fornecer templates reutilizáveis

☒ Demonstrar padrões de projeto na prática

☒ Preparar para desafios reais do mercado

1.3 Público-Alvo Desenvolvedores iniciantes a intermediários

Profissionais buscando padronização

Equipes que desejam adotar boas práticas

2. FUNDAMENTOS DO DESENVOLVIMENTO PROFISSIONAL {#fundamentos} 2.1 Os Pilares do Desenvolvimento de Elite

PILARES FUNDAMENTAIS:  SOLID: - S: Single Responsibility (Uma classe, uma responsabilidade) - O: Open/Closed (Aberto para extensão, fechado para modificação) - L: Liskov Substitution (Subtipos devem ser substituíveis) - I: Interface Segregation (Interfaces específicas) - D: Dependency Inversion (Dependa de abstrações, não de concretos)

 DRY: - Don't Repeat Yourself (Não repita código) - Cada conhecimento deve ter uma representação única

 KISS: - Keep It Simple, Stupid (Mantenha simples) - Simplicidade sobre complexidade

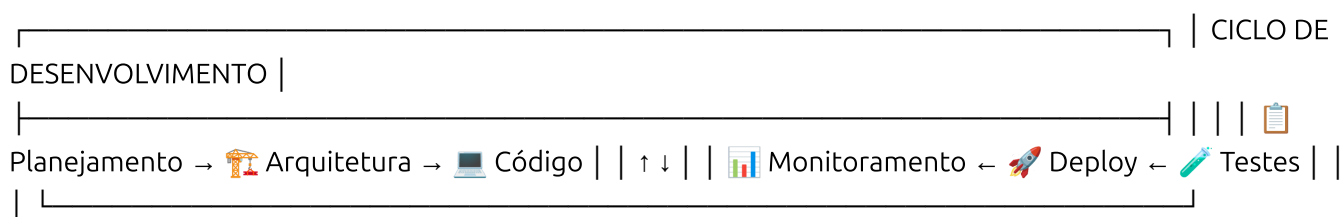
📌 YAGNI: - You Aren't Gonna Need It (Você não vai precisar disso) - Não implemente o que não é necessário agora

📌 DDD: - Domain-Driven Design (Design orientado ao domínio) - Foco no negócio e na linguagem ubíqua

2.2 Mentalidade do Desenvolvedor 10x

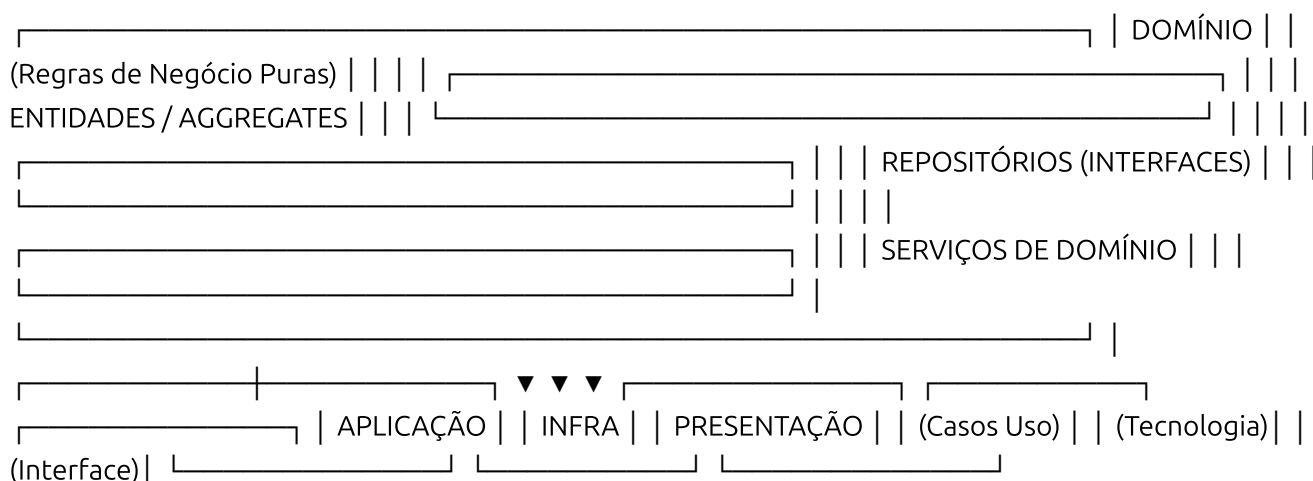
🎯 CARACTERÍSTICAS DE UM DEV 10x: ☐ Pensa em arquitetura antes de codificar ☐ Escreve testes automaticamente ☐ Documenta decisões importantes ☐ Refatora constantemente ☐ Ensina e compartilha conhecimento ☐ Pensa em escalabilidade ☐ Considera segurança desde o início ☐ Automatiza tarefas repetitivas ☐ Mantém-se atualizado ☐ Contribui com a comunidade

2.3 O Ciclo de Desenvolvimento Profissional



3. ARQUITETURA DE SOFTWARE {#arquitetura} 3.1 Arquitetura Hexagonal (Ports & Adapters) A

Arquitetura Hexagonal é o padrão mais utilizado por grandes empresas.



3.2 Componentes da Arquitetura 3.2.1 Domain Layer (Camada de Domínio) Responsabilidade: Regras de negócio puras, independentes de tecnologia.

```
price = $price; $this->stock = $stock; } public function reduceStock(int $quantity): void { if (!$this->stock->hasEnough($quantity)) { throw new InsufficientStockException(); } $this->stock->reduce($quantity); } } //
Domain/ValueObjects/Money.php namespace Domain\ValueObjects; class Money { public function
__construct( private readonly float $amount, private readonly string $currency = 'BRL' ) { if ($amount < 0)
{throw new InvalidMoneyException();}}public function add(Money $other): Money{if($this->currency !== $other->currency) { throw new CurrencyMismatchException(); } return new Money( $this->amount + $other->amount, $this->currency ); } }
3.2.2 Application Layer (Camada de Aplicação) Responsabilidade: Orquestrar casos de uso, coordenar o domínio. name, new Money($command->price), new Stock($command->stock) ); //
2. Salvar $saved = $this->repository->save($product); // 3. Disparar eventos $this->dispatcher->dispatch(new ProductCreatedEvent($saved)); // 4. Retornar DTO return ProductDTO::fromEntity($saved); } }
3.2.3
Infrastructure Layer (Camada de Infraestrutura) Responsabilidade: Implementações concretas de tecnologia (banco, cache, filas). $product->id, [ 'name' => $product->name, 'price' => $product->price->amount(), 'stock'
```

```
=> $product->stock->quantity(), 'currency' => $product->price->currency() ] ); return $product->withId($model->id); } public function findById(int $id): ?Product { $model = EloquentProduct::find($id); if (!$model) { return null; } return new Product( $model->name, new Money($model->price, $model->currency), new Stock($model->stock) )->withId($model->id); } }
```

3.2.4 Presentation Layer (Camada de Apresentação)

Responsabilidade: Interface com o usuário (API, Web, CLI).

3.3 Comparação de Arquiteturas

Aspecto	Hexagonal MVC	Clean Architecture	Independência	Alta	Baixa	Alta
Testabilidade	Excelente	Limitada	Excelente	Média	Baixa	Alta
Complexidade	Média	Baixa	Alta	Curva de Aprendizagem	Média	
Baixa	Alta	Manutenibilidade	Excelente	Média	Excelente	Popularidade
Alta	Muito	Alta	Alta	Alta	Alta	Alta

4. ESTRUTURA DE PROJETOS {#estrutura}

4.1 Estrutura Universal (Multi-Linguagem)

```

project-name/ |
├── src/ # Código fonte
├── Domain/ # Camada de Domínio
├── Entities/ # Entidades de negócio
├── ValueObjects/ # Objetos de valor
├── Aggregates/ # Agregados
├── Events/ # Eventos de domínio
├── Exceptions/ # Exceções de domínio
├── Repositories/ # Interfaces de repositórios
├── Application/ # Camada de Aplicação
├── Commands/ # Comandos (CQRS)
├── Queries/ # Consultas (CQRS)
├── Handlers/ # Manipuladores
├── DTOs/ # Data Transfer Objects
├── Interfaces/ # Interfaces de serviço
├── Services/ # Serviços de aplicação
├── Infrastructure/ # Camada de Infraestrutura
├── Database/ # Configuração DB
├── Repositories/ # Implementações
├── Cache/ # Cache
├── Queue/ # Filas
├── Services/ # Serviços externos
├── Providers/ # Injeção de dependência
├── Presentation/ # Camada de Apresentação
├── Controllers/ # Controladores
├── Middleware/ # Middleware
├── Requests/ # Validações
├── Resources/ # Transformadores
├── Views/ # Templates
├── tests/ # Testes
├── Unit/ # Testes Unitários
├── Integration/ # Testes de Integração
├── Feature/ # Testes Funcionais
├── E2E/ # Testes End-to-End
├── docs/ # Documentação
├── api/ # API Docs
├── architecture/ # ADR
├── guides/ # Guias
├── scripts/ # Scripts de automação
├── docker/ # Docker configurações
├── .github/workflows/ # CI/CD
├── .env.example # Variáveis de ambiente
├── docker-compose.yml # Docker Compose
├── Dockerfile # Dockerfile
├── Makefile # Comandos automáticos
├── README.md # Documentação inicial
├── CHANGELOG.md # Histórico de mudanças
├── LICENSE # Licença

```

4.2 Estrutura Laravel (Seu Projeto Atual)

```

app/
├── Actions/ # → Application/Commands
├── Admin/Produto/
├── AjustarEstoqueAction.php
├── AtualizarProdutoAction.php
├── CriarProdutoAction.php
├── DeletarProdutoAction.php
├── DTOs/ # → Application/DTOs
├── Requests/
├── CreateProductRequestDTO.php
├── UpdateProductRequestDTO.php
├── Responses/
├── ApiResponseDTO.php
├── ProductResponseDTO.php
├── PaymentResponseDTO.php
├── Models/ # → Domain/Entities
├── Produto.php
├── Pedido.php
├── User.php
├── Banner.php
├── Repositories/ # → Infrastructure/Repositories
├── Contracts/ # → Domain/Repositories
├── ProdutoRepositoryInterface.php
├── UserRepositoryInterface.php
├── ProdutoRepository.php
├── Services/ # → Application/Services
├── Contracts/ # → Application/Interfaces
├── ProductServiceInterface.php
├── PaymentServiceInterface.php
├── ProductService.php
├── Http/Controllers/ # → Presentation/Controllers
├── Admin/
├── Api/
├── Auth/
├── Http/Requests/ # → Presentation/Requests
├── Http/Resources/ # → Presentation/Resources
├── Http/Middleware/ # → Presentation/Middleware

```

5. PADRÕES DE DESIGN {#padroes}

5.1 Repository Pattern

Propósito: Abstrair a camada de dados.

```

model->find($id); return $record ? $this->toDomain($record) : null; } public function save(Product $product): Product { $record = $this->model->updateOrCreate( ['id' => $product->id], $this->toPersistence($product) ); return $this->toDomain($record); } private function toDomain(ProductModel

```

```

$record): Product { return new Product( $record->id, $record->name, new Money($record->price), new
Stock($record->stock) ); } private function toPersistence(Product $product): array { return [ 'name' =>
$product->name, 'price' => $product->price->amount(), 'stock' => $product->stock->quantity(), ]; } } 5.2
Service Pattern Propósito: Encapsular lógica de negócio complexa. validateProductData($dto); // 2. Criar
entidade $product = new Product( $dto->name, new Money($dto->price), new Stock($dto->stock) ); // 3.
Salvar $saved = $this->repository->save($product); // 4. Disparar eventos $this->dispatcher->dispatch( new
ProductCreatedEvent($saved) ); // 5. Retornar DTO return ProductDTO::fromEntity($saved); } public function
adjustStock(int $productId, int $quantity): void { $product = $this->repository->find($productId); if
(!$product) { throw new ProductNotFoundException(); } $this->stockService->adjust($product, $quantity);
$this->repository->save($product); } } 5.3 DTO Pattern Propósito: Transferir dados entre camadas. id, name:
$product->name, price: $product->price->amount(), stock: $product->stock->quantity(), description:
$product->description, createdAt: $product->createdAt?->format('Y-m-d H:i:s') ); } public function toArray():
array { return [ 'id' => $this->id, 'name' => $this->name, 'price' => $this->price, 'stock' => $this->stock,
'description' => $this->description, 'created_at' => $this->createdAt ]; } } 5.4 Factory Pattern Propósito: Criar
objetos complexos. reduceStock(3); // Assert $this->assertEquals(7, $product->stock->quantity()); } /** @test
*/ public function it_throws_exception_when_insufficient_stock() { $this-
>expectException(InsufficientStockException::class); $product = new Product( 'Test Product', new
Money(100.00), new Stock(5) ); $product->reduceStock(10); } } 6.3 Teste de Integração repository =
app(ProductRepositoryInterface::class); /** @test */ public function it_can_save_and_find_product() { //
Arrange $product = new Product( 'Test Product', new Money(100.00), new Stock(10) ); // Act $saved = $this-
>repository->save($product); $found = $this->repository->find($saved->id); // Assert $this-
>assertEquals($saved->id, $found->id); $this->assertEquals('Test Product', $found->name); $this-
>assertEquals(100.00, $found->price->amount()); } } 6.4 Teste Funcional (Feature) admin = User::factory()-
>admin()->create(); /** @test */ public function admin_can_create_product() { // Arrange $this-
>actingAs($this->admin); $data = [ 'name' => 'New Product', 'price' => 199.90, 'stock' => 20 ]; // Act $response
= $this->postJson('/api/products', $data); // Assert $response->assertStatus(201) -
>assertJsonPath('data.name', 'New Product'); $this->assertDatabaseHas('products', [ 'name' => 'New
Product', 'price' => 199.90 ]; } } 7. DEVOPS E CI/CD {#devops} 7.1 Docker Setup Completo # Dockerfile (Multi-
stage) FROM php:8.2-fpm AS builder WORKDIR /var/www/html # Install dependencies RUN apt-get update
&& apt-get install -y \ git curl libpng-dev libonig-dev libxml2-dev zip unzip \ && docker-php-ext-install
pdo_mysql mbstring exif pcntl bcmath gd # Install Composer COPY --from=composer:latest
/usr/bin/composer /usr/bin/composer # Copy application COPY . . # Install dependencies (production) RUN
composer install --no-interaction --optimize-autoloader --no-dev # Stage 2: Nginx FROM nginx:alpine COPY --
from=builder /var/www/html /var/www/html COPY docker/nginx/conf.d /etc/nginx/conf.d EXPOSE 80 CMD
["nginx", "-g", "daemon off;"] 7.2 GitHub Actions CI/CD # .github/workflows/deploy.yml name: Deploy to
Production on: push: branches: [ main ] jobs: deploy: runs-on: ubuntu-latest steps: - uses:
actions/checkout@v3 - name: Setup PHP uses: shivammathur/setup-php@v2 with: php-version: '8.2'
extensions: mbstring, intl, pdo_mysql - name: Install Dependencies run: composer install --no-interaction --
optimize-autoloader - name: Run Tests run: | php artisan migrate --force php artisan test - name: Deploy to
Production env: DEPLOY_KEY: ${ secrets.DEPLOY_KEY } DEPLOY_HOST: ${ secrets.DEPLOY_HOST } run: |
echo "$DEPLOY_KEY" > deploy_key chmod 600 deploy_key ssh -i deploy_key $DEPLOY_HOST "cd
/var/www/app && git pull && composer install && php artisan migrate --force" 8. DOCUMENTAÇÃO
{#documentacao} 8.1 README.md Template # 🚀 Nome do Projeto > Breve descrição do projeto ## 📋
Índice - [Sobre](#sobre) - [Tecnologias](#tecnologias) - [Pré-requisitos](#pre-requisitos) - [Instalação]
(#instalação) - [Configuração](#configuração) - [Uso](#uso) - [Testes](#testes) - [Deploy](#deploy) -
[Contribuição](#contribuição) - [Licença](#licença) ## 📖 Sobre Descrição detalhada do projeto. ## 🛠️

```

Tecnologias - ****Backend:**** PHP 8.2, Laravel 10 - ****Frontend:**** Vue.js 3, TailwindCSS - ****Database:**** MySQL 8.0, Redis - ****DevOps:**** Docker, GitHub Actions ## 📋 Pré-requisitos - PHP 8.2+ - Composer - Node.js 18+ - NPM/Yarn - MySQL 8.0+ - Docker (opcional) ## ⚙️ Instalação `\\`\\`bash # Clone o repositório git clone https://github.com/yourusername/project.git cd project # Instale as dependências PHP composer install # Instale as dependências Node npm install # Configure o ambiente cp .env.example .env # Gere a chave php artisan key:generate # Execute as migrações php artisan migrate --seed # Inicie o servidor php artisan serve \\`\\` ## 🧪 Testes \\`\\`bash # Execute todos os testes php artisan test # Execute testes unitários php artisan test --testsuite=Unit # Execute testes de integração php artisan test --testsuite=Integration # Execute testes funcionais php artisan test --testsuite=Feature \\`\\` ## 📄 Licença Este projeto está sob a licença MIT.`

8.2 Architecture Decision Record (ADR) Template

ADR-001: Nome da Decisão ## Status [Proposto | Aceito | Rejeitado | Depreciado] ## Contexto Descrição do contexto e problema. ## Decisão A decisão tomada e o racional por trás dela. ## Consequências ### Positivas - Ponto positivo 1 - Ponto positivo 2 ### Negativas - Ponto negativo 1 - Ponto negativo 2 ## Alternativas Consideradas - Alternativa 1: Motivo da rejeição - Alternativa 2: Motivo da rejeição ## Referências - Link para documentação - Link para discussões

9. PASSO A PASSO PRÁTICO {#pratico}

9.1 Criando um Projeto do Zero

Passo 1: Planejamento 📋

PLANEJAMENTO DO PROJETO

- **Definir o Escopo****
 - Quais funcionalidades principais?
 - Quem será o usuário?
 - Qual o objetivo de negócio?
- **Escolher Tecnologias****
 - Backend: Laravel, Node.js, Python?
 - Frontend: React, Vue, Angular?
 - Database: MySQL, PostgreSQL, MongoDB?
 - Cache: Redis, Memcached?
 - Queue: RabbitMQ, Redis?
- **Definir Arquitetura****
 - Hexagonal, Clean, MVC?
 - Padrões de design?
 - Estrutura de pastas?

Passo 2: Setup Inicial

1. Criar estrutura de pastas `mkdir meu-ecommerce cd meu-ecommerce`
2. Inicializar Git `git init git checkout -b develop git branch main`
3. Criar estrutura base `mkdir -p src/{Domain,Application,Infrastructure,Presentation,Shared} mkdir -p tests/{Unit,Integration,Feature,E2E} mkdir -p docs/{api,architecture,guides} mkdir -p docker/{mysql,nginx,redis} mkdir -p .github/workflows`
4. Criar arquivos base `touch README.md touch .gitignore touch docker-compose.yml touch Dockerfile touch Makefile touch .env.example touch phpunit.xml`

Passo 3: Configurar Dependências

- Laravel** `composer create-project laravel/laravel .`
- Node.js** `npm init -y npm install express mongoose dotenv`
- Python** `python -m venv venv source venv/bin/activate pip install fastapi sqlalchemy alembic`

Passo 4: Implementar Camadas //

- 1. Domain Layer**
 - `src/Domain/Entities/Product.php`
 - `src/Domain/ValueObjects/Money.php`
 - `src/Domain/Repositories/ProductRepositoryInterface.php`
- 2. Application Layer**
 - `src/Application/Commands/CreateProductCommand.php`
 - `src/Application/Handlers/CreateProductHandler.php`
 - `src/Application/DTOs/ProductDTO.php`
- 3. Infrastructure Layer**
 - `src/Infrastructure/Repositories/ProductRepository.php`
 - `src/Infrastructure/Database/Migrations/...`
- 4. Presentation Layer**
 - `src/Presentation/Controllers/ProductController.php`
 - `src/Presentation/Requests/CreateProductRequest.php`

Passo 5: Configurar Testes

- Laravel** `php artisan make:test ProductServiceTest --unit php artisan make:test ProductControllerTest`
- Node.js** `npm install --save-dev jest supertest`
- Python** `pip install pytest pytest-asyncio httpx`

9.2 Comandos Úteis (Makefile)

Makefile

```
.PHONY: help install test migrate seed dev build help: @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) | sort | awk 'BEGIN {FS = ":.*?## "}; {printf "\033[36m%-30s\033[0m %s\n", $$1, $$2}'
```

install: ## Instalar dependências `composer install npm install`

migrate: ## Executar migrações `php artisan migrate`

seed: ## Executar seeders `php artisan db:seed`

test: ## Executar testes `php artisan test`

dev: ## Iniciar servidor de desenvolvimento `php artisan serve`

build: ## Build para produção `composer install --no-dev --optimize-autoloader npm run build`

docker-up: ## Iniciar containers Docker `docker-compose up -d`

docker-down: ## Parar containers Docker `docker-compose down`

logs: ## Ver logs `tail -f storage/logs/laravel.log`

9.3 Template de Configuração (.env)

```
# .env.example APP_NAME=MeuEcommerce APP_ENV=local APP_KEY= APP_DEBUG=true APP_URL=http://localhost:8000 DB_CONNECTION=mysql
```


DB_HOST=127.0.0.1 DB_PORT=3306 DB_DATABASE=ecommerce DB_USERNAME=root DB_PASSWORD=

CACHE_DRIVER=redis SESSION_DRIVER=redis QUEUE_CONNECTION=redis REDIS_HOST=127.0.0.1

REDIS_PASSWORD=null REDIS_PORT=6379 # APIs MERCADOPAGO_PUBLIC_KEY=

MERCADOPAGO_SECRET_KEY= # Email MAIL_MAILER=smtp MAIL_HOST=smtp.mailtrap.io

MAIL_PORT=2525 MAIL_USERNAME=null MAIL_PASSWORD=null 10. CHECKLISTS E TEMPLATES {#checklists}

10.1 Checklist de Qualidade  CHECKLIST DE QUALIDADE DO PROJETO  CHECKLIST DE QUALIDADE DO PROJETO ## Arquitetura ☐ Arquitetura definida e documentada ☐ Separação de camadas (Domain, Application, Infrastructure, Presentation) ☐ SOLID aplicado ☐ Design Patterns apropriados ## Código ☐ Coding Standards definidos ☐ Code Review implementado ☐ Nomenclatura clara ☐ Comentários úteis ☐ Sem duplicação de código (DRY) ## Testes ☐ Testes Unitários (Cobertura > 80%) ☐ Testes de Integração ☐ Testes Funcionais ☐ Testes E2E ☐ CI/CD com testes automáticos ## Documentação ☐ README.md completo ☐ Documentação da API ☐ ADR (Architecture Decision Records) ☐ Guia de contribuição ☐ CHANGELOG.md ## DevOps ☐ Docker configurado ☐ CI/CD implementado ☐ Logging implementado ☐ Monitoring configurado ☐ Backup strategy definida ## Segurança ☐ CORS configurado ☐ CSRF protection ☐ XSS prevention ☐ SQL Injection prevention ☐ Rate Limiting ☐ Authentication/Authorization ☐ Environment variables seguras ☐ Dependências atualizadas ## Performance ☐ Cache implementado ☐ Queue para tarefas pesadas ☐ Database indexes otimizados ☐ Assets otimizados ☐ Lazy loading implementado ☐ Paginação implementada 10.2 Template de Pull Request ##  Descrição - O que foi feito? - Por que foi feito? - Como foi feito? ##  Testes - [] Testes unitários - [] Testes de integração - [] Testes manuais ##  Checklist - [] Código segue padrões do projeto - [] Documentação atualizada - [] Testes passando - [] Sem warnings - [] Sem breaking changes - [] Performance verificada ##  Screenshots (se aplicável) (Coloque screenshots aqui) ##  Links Relacionados - [Link para ticket] - [Link para documentação] 10.3 Template de API Response { "success": true, "data": { "id": 1, "name": "Produto Exemplo", "price": 99.90, "stock": 20 }, "message": "Produto criado com sucesso", "errors": null, "meta": { "timestamp": "2024-01-01T00:00:00.000Z", "request_id": "abc-123" } } 10.4 Template de Exception Handler validationError(\$e); if (\$e instanceof DomainException) { return \$this->domainError(\$e); } if (\$e instanceof ModelNotFoundException) { return \$this->notFoundError(\$e); } if (\$e instanceof AuthenticationException) { return \$this->unauthorizedError(\$e); } return \$this->genericError(\$e); } private function validationError(ValidationException \$e): JsonResponse { return response()->json(['success' => false, 'message' => 'Validation error', 'errors' => \$e->errors(),], 422); } private function domainError(DomainException \$e): JsonResponse { return response()->json(['success' => false, 'message' => \$e->getMessage(),], 400); } } 11. APRENDENDO MAIS 11.1 Recursos Recomendados  Livros Domain-Driven Design - Eric Evans Clean Architecture - Robert C. Martin Implementing DDD - Vaughn Vernon Design Patterns - Gang of Four Refactoring - Martin Fowler  Cursos Clean Architecture (Pluralsight) Domain-Driven Design (Udemy) Test-Driven Development (Laracasts) Docker Mastery (Udemy)  Podcasts Software Engineering Daily Code with Jason Laravel News Podcast The Changelog  Blogs Martin Fowler Blog David Heinemeier Hansson (DHH) Kent Beck Laravel News 11.2 Comunidades GitHub Stack Overflow Reddit (r/laravel, r/programming) Discord/Slack groups Meetups locais 11.3 Ferramentas Essenciais EDITORES: - VS Code (com extensões) - PHPStorm - Sublime Text VERSIONAMENTO: - Git - GitHub/GitLab/Bitbucket TESTES: - PHPUnit/Pest - Jest/Mocha - Pytest CI/CD: - GitHub Actions - GitLab CI - Jenkins CONTAINERS: - Docker - Docker Compose - Kubernetes MONITORING: - Sentry - New Relic - Datadog 12. EXERCÍCIOS PRÁTICOS 12.1 Exercício 1: Criar um CRUD Completo Desafio: Implementar um CRUD de produtos usando a arquitetura hexagonal. Tarefas: Criar a entidade Product (Domain) Criar o repositório (Infrastructure) Criar o service (Application) Criar o controller (Presentation) Escrever testes para cada camada 12.2 Exercício 2: Implementar Pagamento Desafio: Implementar sistema de pagamento com múltiplos gateways. Tarefas: Criar interface PaymentGateway (Domain) Implementar MercadoPagoGateway (Infrastructure) Implementar PayPalGateway (Infrastructure) Criar PaymentService (Application) Criar PaymentController (Presentation)

12.3 Exercício 3: Sistema de Notificações Desafio: Implementar sistema de notificações assíncronas. Tarefas: Criar evento OrderPlaced (Domain) Criar listeners para email e SMS Configurar queue (Infrastructure) Criar testes de integração 12.4 Exercício 4: Refatoração Desafio: Refatorar um projeto existente. Tarefas:

Identificar violações de SOLID Extrair interfaces Separar camadas Escrever testes Documentar mudanças 13.

CONCLUSÃO 13.1 Pontos-Chave Arquitetura é fundamental - Defina antes de codificar Testes são essenciais -

Escreva testes automaticamente Documentação é crucial - Documente decisões DevOps é necessário -

Automatize o deploy Aprendizado contínuo - Nunca pare de estudar 13.2 Próximos Passos ☒ Escolha um

projeto para praticar ☒ Aplique a arquitetura aprendida ☒ Escreva testes para tudo ☒ Configure CI/CD ☒

Documente cada decisão ☒ Compartilhe seu conhecimento 13.3 Mensagem Final "O conhecimento é a

única coisa que cresce quando é compartilhado." — Osho Continue estudando, praticando e compartilhando!

ANEXOS Anexo A: Referências Rápidas Princípios SOLID Cheat Sheet S - Single Responsibility "Uma classe

deve ter um, e somente um, motivo para mudar." O - Open/Closed "Entidades devem estar abertas para

extensão, fechadas para modificação." L - Liskov Substitution "Subtipos devem ser substituíveis por seus

tipos base." I - Interface Segregation "Muitas interfaces específicas são melhores do que uma interface

geral." D - Dependency Inversion "Dependa de abstrações, não de implementações concretas." Padrões de

Design Cheat Sheet CRIACIONAIS: - Factory: Cria objetos sem expor lógica - Singleton: Uma única instância -

Builder: Constrói objetos complexos ESTRUTURAIS: - Adapter: Converte interfaces - Decorator: Adiciona

funcionalidades - Facade: Simplifica interfaces COMPORTAMENTAIS: - Strategy: Algoritmos intercambiáveis -

Observer: Notifica mudanças - Command: Encapsula requisições Anexo B: Comandos Úteis # Git git init git

add . git commit -m "message" git branch git checkout -b feature git merge git rebase # Docker docker build -

t app . docker-compose up -d docker-compose down docker logs app docker exec -it app bash # Laravel php

artisan make:model Product -m php artisan make:controller ProductController --resource php artisan

make:test ProductTest --unit php artisan migrate php artisan tinker php artisan optimize # NPM npm init -y

npm install package-name npm run dev npm run build # Composer composer require package-name

composer install composer update composer dump-autoload Você é um arquiteto de software especialista

em desenvolvimento de sistemas escaláveis, com mais de 15 anos de experiência em grandes empresas de

tecnologia. Sua especialidade é criar projetos com arquitetura hexagonal (ports & adapters), aplicando DDD,

SOLID, TDD e as melhores práticas do mercado. BASEADO NA CONVERSA ANTERIOR SOBRE ARQUITETURA

DE SOFTWARE, CRIE: ## 1. ESTRUTURA DE PASTAS COMPLETA Crie a estrutura de pastas completa para um

projeto [NOME DO PROJETO] usando a arquitetura hexagonal, incluindo: #### Domain Layer - Entities

(entidades de negócio) - Value Objects (objetos de valor) - Aggregates (agregados) - Domain Events (eventos

de domínio) - Domain Exceptions (exceções de domínio) - Repository Interfaces (interfaces de repositório) -

Domain Services (serviços de domínio) #### Application Layer - Commands (CQRS - comandos) - Queries

(CQRS - consultas) - Handlers (manipuladores) - DTOs (Data Transfer Objects) - Application Services (serviços

de aplicação) - Interfaces (contratos) - Use Cases (casos de uso) #### Infrastructure Layer - Database

(ORM/ODM) - Repositories (implementações) - External Services (serviços externos) - Cache (cache) - Queue

(filas) - Logging (logs) - Providers (injeção de dependência) #### Presentation Layer - Controllers

(controladores) - Middleware (middleware) - Requests (validação) - Resources (transformação) - Routes

(rotas) - Views (templates) #### Shared Layer - Helpers (utilitários) - Traits (traits/mixins) - Constants

(constantes) - Config (configurações) #### Tests - Unit (testes unitários) - Integration (testes de integração) -

Feature (testes funcionais) - E2E (testes end-to-end) #### Docs - API (documentação da API) - Architecture

(ADR - Architecture Decision Records) - Guides (guias do desenvolvedor) #### DevOps - Docker (Dockerfile,

docker-compose) - CI/CD (GitHub Actions, GitLab CI) - Scripts (scripts de automação) ## 2. CÓDIGO

COMPLETO PARA CADA CAMADA Para CADA CAMADA, forneça código completo e funcional com: #### 2.1

Domain Layer - [] Entidades com validação de negócio - [] Value Objects com validação - [] Agregados com

invariantes - [] Eventos de domínio - [] Exceções de domínio - [] Interfaces de repositório - [] Serviços de

domínio ### 2.2 Application Layer - [] Commands com validação - [] Queries com filtros - [] Handlers com lógica de orquestração - [] DTOs com transformação - [] Application Services - [] Casos de uso completos

2.3 Infrastructure Layer - [] Implementações de repositório - [] Configuração de banco de dados - [] Serviços externos - [] Cache - [] Filas - [] Logging - [] Injeção de dependência ### 2.4 Presentation Layer - [] Controllers com injeção de dependência - [] Middleware - [] Requests com validação - [] Resources com transformação - [] Rotas ## 3. PADRÕES DE CÓDIGO Crie um guia de padrões de código incluindo: ### 3.1 Nomenclatura - Classes: PascalCase - Methods: camelCase - Variables: camelCase - Constants: UPPER_SNAKE_CASE - Interfaces: [Nome]Interface ou I[Nome] - Abstract Classes: [Nome]Abstract ou Abstract[Nome] ### 3.2 Estrutura de Arquivos - 1 classe por arquivo - Organização por domínio/camada - Namespace/Module organization ### 3.3 Documentação - PHPDoc/JavaDoc/TSDoc - README.md - CONTRIBUTING.md - CHANGELOG.md ## 4. TESTES Crie uma suíte completa de testes: ### 4.1 Testes Unitários (Cobertura > 80%) - Testes de entidades - Testes de value objects - Testes de serviços - Testes de handlers - Mock de dependências ### 4.2 Testes de Integração - Testes de repositórios - Testes de API - Testes de banco de dados - Testes de serviços externos ### 4.3 Testes Funcionais - Testes de endpoints - Testes de autenticação - Testes de autorização - Testes de fluxos completos ## 5. DOCUMENTAÇÃO Crie documentação completa: ### 5.1 README.md - Descrição do projeto - Tecnologias - Pré-requisitos - Instalação - Configuração - Uso - Testes - Deploy - Contribuição - Licença ### 5.2 API Documentation - Endpoints - Request/Response - Authentication - Error codes - Examples ### 5.3 Architecture Decision Records - Contexto - Decisão - Consequências - Alternativas ## 6. CONFIGURAÇÕES DE AMBIENTE ### 6.1 Docker - Dockerfile (multistage) - docker-compose.yml - .dockerignore ### 6.2 CI/CD - GitHub Actions workflows - Deploy scripts - Test automation ### 6.3 Environment Variables - .env.example - Config files ## 7. EXEMPLOS PRÁTICOS CRIE UM CRUD COMPLETO DE [ENTIDADE PRINCIPAL] DEMONSTRANDO: ### 7.1 Criar (Create) - Request validation - Domain entity creation - Repository save - Response transformation ### 7.2 Ler (Read) - Query with filters - Repository find - Response with pagination ### 7.3 Atualizar (Update) - Request validation - Domain entity update - Repository update - Response transformation ### 7.4 Deletar (Delete) - Domain validation - Repository delete - Response ## 8. CHECKLIST DE QUALIDADE Crie um checklist completo para garantir a qualidade do projeto: - [] Arquitetura definida - [] SOLID aplicado - [] Testes implementados - [] Documentação completa - [] CI/CD configurado - [] Docker configurado - [] Security implementado - [] Performance otimizada - [] Code review setup - [] Monitoring configurado - [] Logging implementado - [] Backup strategy ## 9. INSTRUÇÕES FINAIS O código deve ser: - ☒ Funcional e executável - ☒ Bem comentado - ☒ Seguindo padrões de nomenclatura - ☒ Com testes - ☒ Documentado A estrutura deve ser: - ☒ Modular - ☒ Escalável - ☒ Mantível - ☒ Testável - ☒ Independente de tecnologia LINGUAGEM: [PHP, Node.js, Python, Java, etc] FRAMEWORK: [Laravel, Express, FastAPI, Spring Boot, etc] PROJETO: [E-commerce, API REST, Microserviço, etc] ENTIDADE PRINCIPAL: [Product, User, Order, etc] FORNEÇA O CÓDIGO COMPLETO, ORGANIZADO E PRONTO PARA USO. 📄 COMO USAR ESTE PROMPT 1. Substitua os Placeholders LINGUAGEM: [PHP, Node.js, Python, Java, etc] FRAMEWORK: [Laravel, Express, FastAPI, Spring Boot, etc] PROJETO: [E-commerce, API REST, Microserviço, etc] ENTIDADE PRINCIPAL: [Product, User, Order, etc] 2. Exemplo de Prompt Preenchido LINGUAGEM: PHP FRAMEWORK: Laravel PROJETO: E-commerce ENTIDADE PRINCIPAL: Product 3. Exemplo de Prompt para Node.js LINGUAGEM: Node.js FRAMEWORK: Express + TypeScript PROJETO: API REST ENTIDADE PRINCIPAL: User 4. Exemplo de Prompt para Python LINGUAGEM: Python FRAMEWORK: FastAPI PROJETO: Microserviço ENTIDADE PRINCIPAL: Order 📦 VERSÕES DO PROMPT POR LINGUAGEM 🟢 Versão PHP/Laravel 🟡 Versão Node.js/Express LINGUAGEM: TypeScript 5.0 FRAMEWORK: Express.js + TypeORM PROJETO: API REST ENTIDADE PRINCIPAL: User Use: - TypeScript strict mode - TypeORM ou Prisma - Jest para testes - ESLint + Prettier - Winston para logs - Bull para filas - Redis para cache Versão Python/FastAPI LINGUAGEM: Python 3.11 FRAMEWORK: FastAPI + SQLAlchemy PROJETO: Microserviço ENTIDADE PRINCIPAL: Order Use: -

Python 3.11 com type hints - FastAPI com OpenAPI - SQLAlchemy 2.0 - Pytest para testes - Black + Ruff para

code style - Celery para filas - Redis para cache - Alembic para migrações 🟡 Versão Java/Spring Boot

LINGUAGEM: Java 17 FRAMEWORK: Spring Boot 3.0 + JPA PROJETO: Sistema de Vendas ENTIDADE

PRINCIPAL: Customer Use: - Java 17 com records - Spring Boot 3.0 - Spring Data JPA - JUnit 5 + Mockito -

Maven/Gradle - Lombok - Flyway para migrações - Spring Cloud para microserviços 🔧 PROMPT PARA

GERAÇÃO DE MÓDULOS ESPECÍFICOS Módulo de Autenticação BASEADO NA ARQUITETURA DEFINIDA,

CRIE UM MÓDULO COMPLETO DE AUTENTICAÇÃO: 1. Domain Layer - Entity: User - Value Objects: Email,

Password, Token - Interfaces: UserRepositoryInterface, AuthServiceInterface - Events: UserRegistered,

UserLoggedIn 2. Application Layer - Commands: RegisterUserCommand, LoginUserCommand - Handlers:

RegisterUserHandler, LoginUserHandler - DTOs: RegisterUserDTO, LoginUserDTO, AuthResponseDTO 3.

Infrastructure Layer - Repository: UserRepository (JWT/Session) - Services: JWTService, PasswordHasher -

Middleware: AuthMiddleware, RoleMiddleware 4. Presentation Layer - Controllers: AuthController -

Requests: RegisterRequest, LoginRequest - Resources: UserResource, TokenResource 5. Tests - Unit:

AuthServiceTest - Integration: AuthControllerTest - Feature: AuthenticationFlowTest FORNEÇA CÓDIGO

COMPLETO E FUNCIONAL. Módulo de Pagamentos BASEADO NA ARQUITETURA DEFINIDA, CRIE UM

MÓDULO COMPLETO DE PAGAMENTO: 1. Domain Layer - Entities: Payment, Transaction - Value Objects:

Money, PaymentStatus - Interfaces: PaymentGatewayInterface, TransactionRepositoryInterface - Events:

PaymentProcessed, PaymentFailed 2. Application Layer - Commands: ProcessPaymentCommand,

RefundPaymentCommand - Handlers: ProcessPaymentHandler, RefundPaymentHandler - DTOs:

PaymentDTO, TransactionDTO, PaymentResponseDTO 3. Infrastructure Layer - Gateway:

MercadoPagoGateway, PayPalGateway, StripeGateway - Repository: TransactionRepository - Services:

PaymentFactory, WebhookHandler 4. Presentation Layer - Controllers: PaymentController,

WebhookController - Requests: PaymentRequest, WebhookRequest - Resources: PaymentResource 5. Tests -

Unit: PaymentServiceTest - Integration: PaymentGatewayTest - Feature: PaymentFlowTest FORNEÇA

CÓDIGO COMPLETO E FUNCIONAL. 🚀 PROMPT PARA GERAR PROJETO COMPLETO CRIE UM PROJETO

COMPLETO DE [TIPO DE PROJETO] USANDO: LINGUAGEM: [PHP/Node.js/Python/Java] FRAMEWORK:

[Laravel/Express/FastAPI/Spring Boot] ARQUITETURA: Hexagonal + DDD + CQRS TESTES: TDD (Test-Driven

Development) DEVOPS: Docker + CI/CD REQUISITOS FUNCIONAIS: 1. [REQUISITO 1] 2. [REQUISITO 2] 3.

[REQUISITO 3] REQUISITOS TÉCNICOS: 1. Autenticação JWT 2. CRUD completo 3. Cache com Redis 4. Filas

com [RabbitMQ/Redis] 5. Logs estruturados 6. Monitoramento 7. API RESTful 8. Documentação OpenAPI

ENTREGÁVEIS: 1. Código fonte completo 2. Testes (unitários, integração, E2E) 3. Documentação (API,

arquitetura, setup) 4. Docker files 5. CI/CD pipelines 6. README completo FORNEÇA TUDO PRONTO PARA

PRODUÇÃO. 📝 COMO USAR ESTES PROMPTS Passo 1: Escolha o Prompt Adequado Use o prompt principal

para projetos completos Use prompts específicos para módulos Use prompts por linguagem para projetos

específicos Passo 2: Substitua os Placeholders Substitua [NOME DO PROJETO] pelo nome real Substitua

[ENTIDADE PRINCIPAL] pela entidade principal Substitua [TIPO DE PROJETO] pelo tipo de projeto Passo 3:

Adicione Detalhes Específicos Adicione requisitos funcionais específicos Adicione regras de negócio Adicione

integrações com sistemas externos Passo 4: Execute o Prompt Cole o prompt em qualquer assistente IA

(ChatGPT, Claude, etc.) Revise o código gerado Adapte conforme necessário 🎯 EXEMPLO DE RESULTADO

ESPERADO Estrutura de Pastas Gerada 📁 meu-ecommerce/ | — 📁 src/ | | — 📁 Domain/ | | — 📁 Entities/ | | | — Product.php | | — 📁 ValueObjects/ | | | — Money.php | | | — Stock.php | | — 📁 Repositories/ | | | — ProductRepositoryInterface.php | | — 📁 Events/ | | | — ProductCreated.php | — 📁 Application/ | | — 📁 Commands/ | | | — CreateProductCommand.php | | — 📁 Handlers/ | | | — CreateProductHandler.php | | — 📁 DTOs/ | | — ProductDTO.php | — 📁 Infrastructure/ | | — 📁 Repositories/ | | | — ProductRepository.php | | — 📁 Database/ | | — Migrations/ | — 📁 Presentation/ | — 📁

```

Controllers/ | | └─ ProductController.php | └─ Requests/ | | └─ CreateProductRequest.php |
└─ Resources/ | └─ ProductResource.php └─ tests/ | └─ Unit/ | | └─ ProductTest.php |
└─ Integration/ | | └─ ProductRepositoryTest.php | └─ Feature/ | └─
ProductControllerTest.php └─ docs/ | └─ api/ | | └─ openapi.yaml | └─ architecture/ |
└─ adr-001.md └─ docker/ | └─ Dockerfile | └─ docker-compose.yml └─ .github/ | └─
workflows/ | └─ tests.yml | └─ deploy.yml └─ README.md └─ .env.example └─ Makefile

```

💡 DICAS PARA MELHORES RESULTADOS 1. Seja Específico Quanto mais detalhes, melhor o resultado Inclua requisitos de negócio Descreva integrações externas 2. Use Exemplos Forneça exemplos de como deve funcionar Inclua casos de uso específicos Descreva fluxos completos 3. Peça Iterações Peça para refatorar partes específicas Peça para adicionar funcionalidades Peça para otimizar performance 4. Revise e Adapte Revise o código gerado Adapte para seu contexto Teste e valide 📖 RECURSOS ADICIONAIS Templates de Prompt por Caso de Uso API REST Completa text CRIE UMA API REST COMPLETA COM: - Autenticação JWT - CRUD de [entidade] - Paginação e filtros - Upload de arquivos - Email notifications - Rate limiting - CORS configurado - Logs estruturados - Documentação OpenAPI - Testes completos Microserviço text CRIE UM MICROSERVIÇO COMPLETO PARA [função]: - Comunicação síncrona (REST/gRPC) - Comunicação assíncrona (RabbitMQ/Kafka) - Service Discovery - Config Server - API Gateway - Circuit Breaker - Distributed Tracing - Health Checks - Metrics - Docker/Kubernetes Sistema de E-commerce text CRIE UM E-COMMERCE COMPLETO: - Catálogo de produtos - Carrinho de compras - Checkout com múltiplos pagamentos - Gestão de pedidos - Gestão de usuários - Wishlist - Avaliações - Sistema de frete - Notificações - Admin panel - Relatórios - SEO - Analytics 🎉

CONCLUSÃO Com estes prompts profissionais, você pode: ✅ Gerar projetos completos em qualquer linguagem ✅ Aplicar arquiteturas modernas (Hexagonal, Clean) ✅ Seguir padrões de mercado (DDD, SOLID, TDD) ✅ Criar código testável e manutenível ✅ Automatizar setup de projetos ✅ Padronizar desenvolvimento em equipe Use estes prompts como seu assistente pessoal de arquitetura de software! 🚀

🚀 PROMPT PROFISSIONAL: Projeto com MCP Prompt Base para Criar Projetos com Model Context Protocol

📋 PROMPT PRINCIPAL Você é um arquiteto de software especialista em desenvolvimento de sistemas escaláveis e integração com IA, com mais de 15 anos de experiência em grandes empresas de tecnologia. Sua especialidade é criar projetos com arquitetura hexagonal (ports & adapters), aplicando DDD, SOLID, TDD e as melhores práticas do mercado, incluindo o Model Context Protocol (MCP) para integração com agentes de IA. BASEADO NA CONVERSA ANTERIOR SOBRE ARQUITETURA DE SOFTWARE E MCP, CRIE: ## 1.

ESTRUTURA DE PASTAS COMPLETA COM MCP Crie a estrutura de pastas completa para um projeto [NOME DO PROJETO] usando a arquitetura hexagonal com integração MCP, incluindo: #### Domain Layer - Entities (entidades de negócio) - Value Objects (objetos de valor) - Aggregates (agregados) - Domain Events (eventos de domínio) - Domain Exceptions (exceções de domínio) - Repository Interfaces (interfaces de repositório) - Domain Services (serviços de domínio) #### Application Layer - Commands (CQRS - comandos) - Queries (CQRS - consultas) - Handlers (manipuladores) - DTOs (Data Transfer Objects) - Application Services (serviços de aplicação) - Interfaces (contratos) - Use Cases (casos de uso) #### Infrastructure Layer - Database (ORM/ODM) - Repositories (implementações) - External Services (serviços externos) - Cache (cache) - Queue (filas) - Logging (logs) - Providers (injeção de dependência) -  MCP (Model Context Protocol) - Servers (servidores MCP) - [Domain]Server (ex: ProductServer, OrderServer) - Tools (ferramentas MCP) - Create[Entity]Tool - List[Entity]Tool - Get[Entity]Tool - Update[Entity]Tool - Delete[Entity]Tool - [CustomAction]Tool - Resources (recursos MCP) - [Entity]Resource - [Entity]CollectionResource - Prompts (prompts MCP) - [UseCase]Prompt - Server.ts (configuração do servidor) - Clients (clientes MCP - opcional) - Gateway (agregação de servidores - opcional) #### Presentation Layer - Controllers (controladores) - Middleware (middleware) - Requests (validação) - Resources (transformação) - Routes (rotas) - Views (templates) #### Shared Layer - Helpers (utilitários) - Traits (traits/mixins) - Constants (constantes) - Config (configurações) #### MCP Global Configuration - schemas/ (schemas JSON para ferramentas) - gateway/

(MCP Gateway) - mcp.config.json (configuração principal) ### Tests - Unit (testes unitários) - Integration (testes de integração) - Feature (testes funcionais) - MCP (testes específicos para MCP) - ToolsTest (testes de ferramentas) - ResourcesTest (testes de recursos) - PromptsTest (testes de prompts) ### Docs - API (documentação da API) - Architecture (ADR - Architecture Decision Records) - Guides (guias do desenvolvedor) -  MCP (documentação MCP) - Tools (lista e descrição das ferramentas) - Resources (lista e descrição dos recursos) - Prompts (lista e descrição dos prompts) - Integration (como integrar com clientes MCP) ### DevOps - Docker (Dockerfile, docker-compose) - CI/CD (GitHub Actions, GitLab CI) - Scripts (scripts de automação) ## 2. CÓDIGO COMPLETO PARA CADA CAMADA Para CADA CAMADA, forneça código completo e funcional com: ### 2.1 Domain Layer - [] Entidades com validação de negócio - [] Value Objects com validação - [] Agregados com invariantes - [] Eventos de domínio - [] Exceções de domínio - [] Interfaces de repositório - [] Serviços de domínio ### 2.2 Application Layer - [] Commands com validação - [] Queries com filtros - [] Handlers com lógica de orquestração - [] DTOs com transformação - [] Application Services - [] Casos de uso completos ### 2.3 Infrastructure Layer - [] Implementações de repositório - [] Configuração de banco de dados - [] Serviços externos - [] Cache - [] Filas - [] Logging - [] Injeção de dependência - []  MCP Servers completos - [] Configuração do servidor - [] Ferramentas MCP (CRUD + Custom) - [] Recursos MCP - [] Prompts MCP - [] Integração com Application Handlers ### 2.4 Presentation Layer - [] Controllers com injeção de dependência - [] Middleware - [] Requests com validação - [] Resources com transformação - [] Rotas ## 3. MCP IMPLEMENTAÇÃO DETALHADA ### 3.1 Configuração do Servidor MCP - [] Server.ts com configuração completa - [] Registro de ferramentas - [] Registro de recursos - [] Registro de prompts - [] Handler de listagem de ferramentas - [] Handler de execução de ferramentas - [] Handler de recursos - [] Handler de prompts ### 3.2 Ferramentas MCP Para CADA ferramenta, forneça: - [] Nome e descrição (clara para LLMs) - [] Schema de entrada (Zod/JSON Schema) - [] Handler que reutiliza Application Handlers - [] Resposta com conteúdo e dados estruturados - [] Tratamento de erros - [] Logging ### 3.3 Recursos MCP Para CADA recurso, forneça: - [] URI e nome do recurso - [] Descrição (clara para LLMs) - [] MIME type - [] Handler que retorna o recurso - [] Cache strategy ### 3.4 Prompts MCP Para CADA prompt, forneça: - [] Nome e descrição do prompt - [] Argumentos (se houver) - [] Template do prompt - [] Handler que gera o prompt ### 3.5 MCP Gateway (Opcional) - [] Agregação de múltiplos servidores - [] Roteamento de requisições - [] Load balancing - [] Autenticação ## 4. EXEMPLO PRÁTICO: MÓDULO [ENTIDADE] COMPLETO CRIE UM MÓDULO COMPLETO DE [ENTIDADE PRINCIPAL] COM MCP: ### 4.1 Domain Layer - [] Entity com validações - [] Value Objects - [] Repository Interface - [] Domain Events ### 4.2 Application Layer - [] Command (Create, Update, Delete) - [] Query (List, Get) - [] Handlers para cada comando/query - [] DTOs ### 4.3 Infrastructure Layer - [] Repository Implementation - [] MCP Server - [] Create[Entity]Tool - [] List[Entity]Tool - [] Get[Entity]Tool - [] Update[Entity]Tool - [] Delete[Entity]Tool - [] [Custom]Tool - [] [Entity]Resource ### 4.4 Presentation Layer - [] REST Controller - [] Requests - [] Resources - [] Routes ### 4.5 Tests - [] Unit Tests - [] Integration Tests - [] MCP Tools Tests - [] MCP Resources Tests ## 5. PADRÕES DE CÓDIGO Crie um guia de padrões de código incluindo: ### 5.1 Nomenclatura - Classes: PascalCase - Methods: camelCase - Variables: camelCase - Constants: UPPER_SNAKE_CASE - Interfaces: [Nome]Interface ou I[Nome] - Abstract Classes: [Nome]Abstract ou Abstract[Nome] -  MCP Tools: [action][Entity]Tool (ex: CreateProductTool) -  MCP Resources: [Entity]Resource (ex: ProductResource) -  MCP Prompts: [action][Entity]Prompt (ex: HelpProductPrompt) ### 5.2 Estrutura de Arquivos - 1 classe por arquivo - Organização por domínio/camada - Namespace/Module organization ### 5.3 Documentação - PHPDoc/JavaDoc/TSDoc - README.md - CONTRIBUTING.md - CHANGELOG.md -  MCP_TOOLS.md (documentação das ferramentas MCP) ## 6. TESTES Crie uma suíte completa de testes: ### 6.1 Testes Unitários (Cobertura > 80%) - Testes de entidades - Testes de value objects - Testes de serviços - Testes de handlers - Mock de dependências ### 6.2 Testes de Integração - Testes de repositórios - Testes de API - Testes de banco de dados - Testes de serviços externos ### 6.3 Testes Funcionais - Testes de endpoints - Testes de autenticação - Testes de

autorização - Testes de fluxos completos ### 6.4  Testes MCP - Testes de ferramentas (chamada direta) - Testes de recursos - Testes de prompts - Testes de integração com cliente MCP - Testes de erro ## 7.

DOCUMENTAÇÃO Crie documentação completa: ### 7.1 README.md - Descrição do projeto - Tecnologias - Pré-requisitos - Instalação - Configuração - Uso (REST + MCP) - Testes - Deploy - Contribuição - Licença ### 7.2 API Documentation - Endpoints REST - Request/Response - Authentication - Error codes - Examples ### 7.3  MCP Documentation (mcp/README.md) - Introdução ao MCP - Como configurar clientes MCP - Lista completa de ferramentas com exemplos - Lista completa de recursos - Lista completa de prompts - Exemplos de uso com Claude Desktop - Exemplos de uso com VS Code Copilot - Troubleshooting ### 7.4 Architecture Decision Records - Contexto - Decisão (incluindo decisões sobre MCP) - Consequências - Alternativas ## 8.

CONFIGURAÇÕES DE AMBIENTE ### 8.1 Docker (com MCP Sidecar) - Dockerfile (multistage) - docker-compose.yml (com serviços MCP sidecar) - .dockerignore ### 8.2 CI/CD - GitHub Actions workflows - Deploy scripts - Test automation (incluindo testes MCP) ### 8.3 Environment Variables - .env.example - Config files -  MCP_* variables ## 9. EXEMPLO DE CONFIGURAÇÃO PARA CLIENTES MCP Forneça exemplos de configuração para: ### 9.1 Claude Desktop ``json { "mcpServers": { "[project-name]": { "command": "node", "args": ["dist/mcp/server.js"], "env": { "DATABASE_URL": "..." } } } } 9.2 VS Code Copilot (Configuração específica) 9.3 Cursor (Configuração específica) 9.4 Client HTTP Exemplo de chamada HTTP para MCP Server

10. CHECKLIST DE QUALIDADE Crie um checklist completo: ☐ Arquitetura definida (Hexagonal + MCP) ☐ SOLID aplicado ☐ Testes implementados ☐ Documentação completa ☐ CI/CD configurado ☐ Docker configurado ☐ Security implementado ☐ Performance otimizada ☐ Code review setup ☐ Monitoring configurado ☐ Logging implementado ☐ Backup strategy ☐  MCP Servers configurados ☐  Ferramentas MCP implementadas ☐  Recursos MCP implementados ☐  Prompts MCP implementados ☐  Documentação MCP completa ☐  Testes MCP implementados ☐  Exemplos de integração MCP

11. INSTRUÇÕES FINAIS O código deve ser: ☒ Funcional e executável ☒ Bem comentado ☒ Seguindo padrões de nomenclatura ☒ Com testes (incluindo MCP) ☒ Documentado A estrutura deve ser: ☒ Modular ☒ Escalável ☒ Mantível ☒ Testável ☒ Independente de tecnologia ☒ ☒ Preparada para integração com IA via MCP LINGUAGEM: [PHP, Node.js, Python, Java, etc] FRAMEWORK: [Laravel, Express, FastAPI, Spring Boot, etc] PROJETO: [E-commerce, API REST, Microserviço, etc] ENTIDADE PRINCIPAL: [Product, User, Order, etc] TRANSPORTE MCP: [stdio, streamable-http, ou ambos] FORNEÇA O CÓDIGO COMPLETO, ORGANIZADO E PRONTO PARA USO. text --- ##  **EXEMPLOS DE PROMPTS PREENCHIDOS** ###

Exemplo 1: Node.js + Express com MCP LINGUAGEM: TypeScript 5.0 FRAMEWORK: Express.js + TypeORM PROJETO: E-commerce ENTIDADE PRINCIPAL: Product TRANSPORTE MCP: stdio e streamable-http Use: TypeScript strict mode TypeORM ou Prisma Jest para testes ESLint + Prettier Winston para logs Bull para filas Redis para cache  @modelcontextprotocol/sdk  Zod para schemas MCP text ### Exemplo 2: PHP/Laravel com MCP LINGUAGEM: PHP 8.2 FRAMEWORK: Laravel 10 PROJETO: API REST + MCP ENTIDADE PRINCIPAL: Product TRANSPORTE MCP: streamable-http Use: PHP 8.2 com typed properties Laravel 10 com Eloquent PHPUnit para testes PHPStan nível 9 para análise estática Laravel Pint para code style Laravel Horizon para filas  FastMCP para integração Laravel  Zod ou JsonSchema para validação text ###

Exemplo 3: Python/FastAPI com MCP LINGUAGEM: Python 3.11 FRAMEWORK: FastAPI + SQLAlchemy PROJETO: Microserviço com MCP ENTIDADE PRINCIPAL: Order TRANSPORTE MCP: streamable-http Use: Python 3.11 com type hints FastAPI com OpenAPI SQLAlchemy 2.0 Pytest para testes Black + Ruff para code style Celery para filas  mcp-sdk Python  Pydantic para schemas text --- ##  **PROMPT PARA GERAR MÓDULO MCP ESPECÍFICO** BASEADO NA ARQUITETURA DEFINIDA, CRIE UM MÓDULO MCP COMPLETO PARA [DOMÍNIO]:

1. SERVIDOR MCP Crie um servidor MCP completo para [DOMÍNIO] que exponha: Ferramentas MCP (CRUD + Custom) Create[Entity] List[Entity] Get[Entity] Update[Entity] Delete[Entity] [CustomAction1] [CustomAction2] Recursos MCP [Entity]Resource (detalhado) [Entity]CollectionResource (listagem) Prompts MCP Help[Entity]Prompt (ajuda para criar) [Custom]Prompt

2. IMPLEMENTAÇÃO Para

CADA ferramenta, forneça: Schema (Zod/Pydantic) Handler (reutilizando Application Handlers) Resposta formatada Tratamento de erros 3. TESTES Forneça testes para: Cada ferramenta Recursos Prompts 4. DOCUMENTAÇÃO Forneça documentação para: Como usar cada ferramenta Exemplos de chamada Exemplos de integração com Claude Desktop LINGUAGEM: [PHP/Node.js/Python/Java] FRAMEWORK: [Laravel/Express/FastAPI/Spring Boot] DOMÍNIO: [Produtos/Pedidos/Usuários] text --- ## 📝 ****COMO USAR ESTES PROMPTS COM MCP**** ### Passo 1: Escolha o Prompt - Use o prompt principal para projetos completos com MCP - Use o prompt específico para módulos MCP - Adapte para sua linguagem/framework ### Passo 2: Substitua Placeholders - `[NOME DO PROJETO]` → Nome do projeto - `[ENTIDADE PRINCIPAL]` → Entidade principal - `[DOMÍNIO]` → Domínio específico - `[LINGUAGEM]` → Linguagem de programação - `[FRAMEWORK]` → Framework utilizado - `[TRANSPORTE MCP]` → stdio, streamable-http, ou ambos ### Passo 3: Execute com um Assistente IA - Cole o prompt em ChatGPT, Claude, ou similar - Revise o código gerado - Adapte conforme necessário ### Passo 4: Teste a Integração MCP ``bash # Teste local com stdio node dist/mcp/server.js # Teste com Claude Desktop (configurar o arquivo) # ~/Library/Application Support/Claude/claude_desktop_config.json 🎯 RESULTADO ESPERADO Com este prompt, você receberá: ☒ Estrutura completa com todas as camadas + MCP ☒ Código funcional para cada camada ☒ MCP Servers configurados com ferramentas, recursos e prompts ☒ Testes completos incluindo testes MCP ☒ Documentação completa incluindo documentação MCP ☒ Configurações Docker e CI/CD ☒ Exemplos de integração com Claude Desktop, VS Code, etc. ☒ Checklist de qualidade incluindo itens MCP 📚 RECURSOS ADICIONAIS SOBRE MCP Documentação Oficial Model Context Protocol - Site Oficial MCP GitHub MCP Specification Ferramentas MCP FastMCP - Framework Python MCP SDK TypeScript MCP SDK Python Exemplos MCP Servers Repository Reference Servers Sample Implementations 🎉 CONCLUSÃO Com este prompt profissional incluindo MCP, você pode: ☒ Gerar projetos completos com integração MCP ☒ Preparar sua aplicação para agentes de IA ☒ Seguir padrões de mercado (DDD, SOLID, TDD, MCP) ☒ Criar código testável e manutenível ☒ Automatizar setup de projetos com MCP ☒ Padronizar desenvolvimento em equipe ☒ Estar pronto para o futuro da interação com IA 🚀 PROMPTS PROFISSIONAIS: Arquitetura de Software com MCP Guia Completo para Qualquer Linguagem 📑 ÍNDICE DE PROMPTS Prompt Base Completo - Para projetos do zero Prompt por Linguagem - Específico para cada stack Prompt por Tipo de Projeto - E-commerce, API, Microserviço Prompt para Módulos MCP - Módulos específicos Prompt para Refatoração - Adicionar MCP a projetos existentes Prompt para Testes MCP - Suíte completa de testes Prompt para Documentação - Documentação profissional Prompt para DevOps com MCP - Docker e CI/CD 1. PROMPT BASE COMPLETO {#prompt-base} text VOCÊ É UM ARQUITETO DE SOFTWARE ESPECIALISTA COM 15+ ANOS DE EXPERIÊNCIA, COM EXPERTISE EM: 1. Arquitetura Hexagonal (Ports & Adapters) 2. Domain-Driven Design (DDD) 3. SOLID, DRY, KISS, YAGNI 4. Test-Driven Development (TDD) 5. Model Context Protocol (MCP) para integração com IA 6. Clean Code e Padrões de Projeto 7. DevOps e CI/CD CRIE UM PROJETO COMPLETO E PRODUÇÃO-READY COM: ## REQUISITOS DO PROJETO ### Informações Básicas - ****Nome do Projeto:**** [NOME_DO_PROJETO] - ****Linguagem:**** [LINGUAGEM] - ****Framework:**** [FRAMEWORK] - ****Tipo de Projeto:**** [TIPO] - ****Entidade Principal:**** [ENTIDADE] #### Funcionalidades Obrigatórias - [] CRUD completo de [ENTIDADE] - [] Autenticação e Autorização - [] Validações de negócio - [] Tratamento de erros - [] Logging estruturado - [] Testes automatizados - [] Documentação da API - [] Integração MCP (Model Context Protocol) ## 1. ESTRUTURA DE PASTAS Crie a estrutura de pastas seguindo a arquitetura hexagonal: 📁 [NOME_DO_PROJETO]/ | — 📁 src/ | | — 📁 Domain/ | | | — 📁 Entities/ | | | — 📁 ValueObjects/ | | | — 📁 Aggregates/ | | | — 📁 Events/ | | | — 📁 Exceptions/ | | | — 📁 Repositories/ # Interfaces | | — 📁 Services/ # Domain Services | | | — 📁 Application/ | | | — 📁 Commands/ | | | — 📁 Queries/ | | | — 📁 Handlers/ | | | — 📁 DTOs/ | | | — 📁 Services/ # Application Services | | — 📁 Interfaces/ | | | — 📁 Infrastructure/ | | | — 📁 Database/ | | | — 📁 Repositories/ # Implementações | | | — 📁 Cache/ | | | — 📁 Queue/ | | | — 📁 Services/ #

External Services | | └─ MCP/ #  Model Context Protocol | | └─ Servers/ | | | └─ [Entity]Server/ | | | └─ Tools/ | | | └─ Resources/ | | | └─ Prompts/ | | | └─ Server.ts | | | └─ Clients/ | | | └─ Gateway/ | | | └─ Presentation/ | | | └─ Controllers/ | | | └─ Middleware/ | | | └─ Requests/ | | | └─ Resources/ | | | └─ Routes/ | | | └─ tests/ | | | └─ Unit/ | | | └─ Integration/ | | | └─ Feature/ | | | └─ MCP/ | | | └─ docs/ | | | └─ api/ | | | └─ architecture/ | | | └─ mcp/ | | | └─ docker/ | | | └─ .github/workflows/ | | | └─ README.md | | | └─ .env.example | | | └─ Makefile text

2. CÓDIGO COMPLETO POR CAMADA ### 2.1 DOMAIN LAYER

Forneça código completo para: ##### Entity ``[LINGUAGEM] // src/Domain/Entities/[Entity].js/ts/php/java/py // Implementação completa com: // - Propriedades privadas // - Getters/Setters // - Validações de negócio // - Métodos de domínio // - Invariantes Value Object [LINGUAGEM] //

src/Domain/ValueObjects/[ValueObject].js/ts/php/java/py // Implementação completa com: // - Imutabilidade // - Validação no construtor // - Métodos de comparação // - Operações aritméticas (se aplicável) Repository Interface [LINGUAGEM] // src/Domain/Repositories/[Entity]RepositoryInterface.js/ts/php/java/py // Métodos: // - find(id) // - findAll(criteria) // - save(entity) // - delete(id) // - find[Custom](params) Domain Event [LINGUAGEM] // src/Domain/Events/[Entity][Action]Event.js/ts/php/java/py // - Propriedades do evento // - Timestamp // - Event name Domain Exception [LINGUAGEM] //

src/Domain/Exceptions/[Custom]Exception.js/ts/php/java/py // - Extensão de Exception base // - Código de erro // - Mensagem customizada 2.2 APPLICATION LAYER Forneça código completo para: Command [LINGUAGEM] // src/Application/Commands/[Action][Entity]Command.js/ts/php/java/py // - Propriedades (readonly/immutable) // - Validação (se aplicável) Handler [LINGUAGEM] // src/Application/Handlers/[Action][Entity]Handler.js/ts/php/java/py // - Injeção de dependências // - Método handle(command) // - Validação // - Orquestração // - Disparo de eventos DTO [LINGUAGEM] //

src/Application/DTOs/[Entity]DTO.js/ts/php/java/py // - Propriedades // - Métodos de fábrica (fromEntity, fromRequest) // - Métodos de transformação (toArray, toJson) Application Service [LINGUAGEM] // src/Application/Services/[Entity]Service.js/ts/php/java/py // - Interface // - Implementação // - Métodos de orquestração // - Transações // - Eventos 2.3 INFRASTRUCTURE LAYER Forneça código completo para: Repository Implementation [LINGUAGEM] //

src/Infrastructure/Repositories/[Entity]Repository.js/ts/php/java/py // - Implementação da interface // - Mapeamento ORM/ODM // - Queries // - Transactions // - Soft Delete (se aplicável) MCP Server [LINGUAGEM] // src/Infrastructure/MCP/Servers/[Entity]Server/Server.js/ts/php/java/py // - Configuração do servidor // - Registro de ferramentas // - Registro de recursos // - Registro de prompts // - Handlers MCP Tool [LINGUAGEM] // src/Infrastructure/MCP/Servers/[Entity]Server/Tools/[Action][Entity]Tool.js/ts/php/java/py // - Schema (Zod/Pydantic/JsonSchema) // - Descrição clara para LLMs // - Handler que reutiliza Application Handlers // - Formatação de resposta // - Tratamento de erros MCP Resource [LINGUAGEM] //

src/Infrastructure/MCP/Servers/[Entity]Server/Resources/[Entity]Resource.js/ts/php/java/py // - URI // - MIME type // - Handler // - Cache (se aplicável) 2.4 PRESENTATION LAYER Forneça código completo para: Controller [LINGUAGEM] // src/Presentation/Controllers/[Entity]Controller.js/ts/php/java/py // - Injeção de handlers // - Métodos REST (CRUD) // - Status codes apropriados // - Transformação de resposta Request [LINGUAGEM] // src/Presentation/Requests/[Action][Entity]Request.js/ts/php/java/py // - Validação de entrada // - Regras de validação // - Mensagens de erro Resource [LINGUAGEM] //

src/Presentation/Resources/[Entity]Resource.js/ts/php/java/py // - Transformação de dados // - Inclusão de relacionamentos // - Formatação de datas Routes [LINGUAGEM] //

src/Presentation/Routes/[Entity]Routes.js/ts/php/java/py // - Definição de rotas // - Middleware // - Validação

3. MCP IMPLEMENTAÇÃO DETALHADA 3.1 Configuração do Servidor [LINGUAGEM] //

src/Infrastructure/MCP/Servers/[Entity]Server/index.js/ts/php/java/py // - Criação do servidor // -

Configuração de transport // - Inicialização // - Tratamento de erros 3.2 Ferramentas MCP Para CADA

ferramenta, forneça: [LINGUAGEM] //

```
src/Infrastructure/MCP/Servers/[Entity]Server/Tools/List[Entity]Tool.js/ts/php/java/py // 1. Schema de
entrada const schema = { page: { type: 'number', optional: true, default: 1 }, limit: { type: 'number', optional:
true, default: 10 }, filter: { type: 'object', optional: true } }; // 2. Definição da ferramenta const tool = { name:
'list_[entity]', title: 'List [Entity]s', description: 'Lista e filtra [entities] com paginação.', inputSchema: schema,
handler: async (input) => { try { const handler = container.get(ListEntitiesHandler); const result = await
handler.handle(new ListEntitiesQuery(input)); return { content: [{ type: 'text', text: formatList(result) }],
structuredData: result }; } catch (error) { return { content: [{ type: 'text', text: `Error: ${error.message}` } ] }; } };
```

3.3 Prompts MCP [LINGUAGEM] //

```
src/Infrastructure/MCP/Servers/[Entity]Server/Prompts/Help[Entity]Prompt.js/ts/php/java/py const prompt
= { name: 'help_[entity]', description: 'Guia para criar/manipular [entities]', arguments: [ { name: 'context',
description: 'Contexto do usuário', required: false } ], handler: async (args) => ({ messages: [ { role: 'user',
content: { type: 'text', text: `Aqui estão as instruções para trabalhar com [entities]: 1. Para criar: use a
ferramenta create_[entity] 2. Para listar: use list_[entity] 3. Para atualizar: use update_[entity] [Mais
instruções...]` } } ] }); 4. TESTES COMPLETOS 4.1 Testes Unitários [LINGUAGEM] //
```

tests/Unit/[Entity]Test.js/ts/php/java/py // - Testes de entidade // - Testes de value objects // - Testes de

serviços // - Testes de handlers // - Mocks de dependências 4.2 Testes de Integração [LINGUAGEM] //

tests/Integration/[Entity]RepositoryTest.js/ts/php/java/py // - Testes de repositório // - Testes de banco // -

Testes com dados reais 4.3 Testes de API [LINGUAGEM] //

tests/Feature/[Entity]ControllerTest.js/ts/php/java/py // - Testes de endpoints // - Testes de autenticação // -

Testes de autorização // - Testes de validação 4.4 Testes MCP [LINGUAGEM] //

tests/MCP/[Entity]ToolsTest.js/ts/php/java/py // - Testes de cada ferramenta // - Testes de recursos // - Testes
de prompts // - Testes de integração 5. DOCUMENTAÇÃO 5.1 README.md markdown # 🚀

[NOME_DO_PROJETO] [Descrição do projeto] ## 📄 Índice - [Sobre](#sobre) - [Tecnologias](#tecnologias) -

[Pré-requisitos](#pre-requisitos) - [Instalação](#instalação) - [Configuração](#configuração) - [Uso (REST)]

(#uso-rest) - [Uso (MCP)](#uso-mcp) - [Testes](#testes) - [Deploy](#deploy) - [Contribuição](#contribuição) -

[Licença](#licença) ## 🛠️ Tecnologias - [LINGUAGEM] - [FRAMEWORK] - [Banco de Dados] - [Cache] - [Filas] -

[MCP] ## 📄 Pré-requisitos - [Lista de pré-requisitos] ## ⚙️ Instalação [Passos de instalação] ## 🐳 Uso

(MCP) ### Configurar no Claude Desktop [Configuração JSON] ### Ferramentas Disponíveis [Lista de

ferramentas com exemplos] ### Recursos Disponíveis [Lista de recursos com exemplos] ### Prompts

Disponíveis [Lista de prompts com exemplos] 5.2 MCP Tools Documentation markdown # 🛠️ MCP Tools -

[NOME_DO_PROJETO] ## create_[entity] **Descrição:** Cria um novo [entity] no sistema. **Parâmetros:** |

Parâmetro | Tipo | Obrigatório | Descrição | |-----|-----|-----|-----| | name | string | Sim | Nome do

[entity] | | ... | ... | ... | ... | **Exemplo:** ``json { "name": "create_[entity]", "arguments": { "name": "Exemplo", ...

}} list_[entities] Descrição: Lista [entities] com filtros e paginação. Parâmetros: Parâmetro Tipo Obrigatório

Descrição page number Não Número da página (default: 1) limit number Não Itens por página (default: 10) ...

... .. Exemplo: json { "name": "list_[entities]", "arguments": { "page": 1, "limit": 10 } } text ## 6. DEVOPS E

CONFIGURAÇÃO ### 6.1 Docker Compose (com MCP Sidecar) ``yaml version: '3.8' services: app: build: .

container_name: [project]-app ports: - "8000:8000" environment: - APP_ENV=production - DB_HOST=db -

REDIS_HOST=redis depends_on: - db - redis networks: - [project]-network mcp-server: build: context: .

dockerfile: Dockerfile.mcp container_name: [project]-mcp ports: - "8001:8000" environment: -

NODE_ENV=production - DATABASE_URL=postgresql://... depends_on: - app networks: - [project]-network

mcp-gateway: image: mcp-gateway:latest container_name: [project]-mcp-gateway ports: - "8002:8000"

environment: - MCP_SERVERS=app:http://mcp-server:8000/mcp networks: - [project]-network db: image:

postgres:15-alpine container_name: [project]-db environment: - POSTGRES_USER=app -

POSTGRES_PASSWORD=secret - POSTGRES_DB=[project] volumes: - postgres_data:/var/lib/postgresql/data

networks: - [project]-network redis: image: redis:7-alpine container_name: [project]-redis networks: - [project]-network networks: [project]-network: driver: bridge volumes: postgres_data: 6.2 GitHub Actions (com MCP Tests) yml # .github/workflows/ci.yml name: CI/CD Pipeline on: push: branches: [main, develop] pull_request: branches: [main] jobs: tests: runs-on: ubuntu-latest services: postgres: image: postgres:15-alpine env: POSTGRES_USER: app POSTGRES_PASSWORD: secret POSTGRES_DB: test ports: - 5432:5432 options: >- --health-cmd pg_isready --health-interval 10s --health-timeout 5s --health-retries 5 redis: image: redis:7-alpine ports: - 6379:6379 options: >- --health-cmd "redis-cli ping" --health-interval 10s --health-timeout 5s --health-retries 5 steps: - uses: actions/checkout@v3 - name: Setup [LINGUAGEM] # Configuração específica da linguagem - name: Install Dependencies run: | # Comando específico da linguagem - name: Run Unit Tests run: | # Comando de testes unitários - name: Run Integration Tests run: | # Comando de testes de integração - name: Run MCP Tests run: | # Comando de testes MCP - name: Run API Tests run: | # Comando de testes de API build-and-deploy: needs: tests runs-on: ubuntu-latest if: github.ref == 'refs/heads/main' steps: - uses: actions/checkout@v3 - name: Build Docker Image run: docker build -t [image]:latest . - name: Push to Registry run: docker push [registry]/[image]:latest - name: Deploy to [Environment] run: | # Comando de deploy

7. CHECKLIST DE QUALIDADE text  CHECKLIST DE QUALIDADE - PROJETO COM MCP ##
 Arquitetura ☐ Arquitetura Hexagonal implementada ☐ DDD aplicado ☐ SOLID respeitado ☐ DRY aplicado ☐ KISS aplicado ## Código ☐ Coding Standards definidos ☐ Code Review implementado ☐ Nomenclatura clara ☐ Comentários úteis ☐ Sem duplicação ## Domain Layer ☐ Entidades com validação ☐ Value Objects imutáveis ☐ Aggregates com invariantes ☐ Domain Events ☐ Domain Exceptions ## Application Layer ☐ Commands e Queries (CQRS) ☐ Handlers ☐ DTOs ☐ Application Services ☐ Use Cases ## Infrastructure Layer ☐ Repository Implementations ☐ External Services ☐ Cache ☐ Queue ☐ Logging ☐  MCP Servers ## Presentation Layer ☐ Controllers REST ☐ Validation ☐ Resources ☐ Routes ##  MCP ☐ MCP Server configurado ☐ Ferramentas MCP (CRUD + Custom) ☐ Recursos MCP ☐ Prompts MCP ☐ MCP Gateway (se aplicável) ☐ Documentação MCP ## Testes ☐ Unit Tests (cobertura > 80%) ☐ Integration Tests ☐ Feature Tests ☐  MCP Tests ☐ E2E Tests ## Documentação ☐ README.md completo ☐ API Documentation ☐  MCP Documentation ☐ Architecture Decision Records ☐ Contributing Guide ☐ CHANGELOG.md ## DevOps ☐ Docker configurado ☐ CI/CD implementado ☐ Monitoring ☐ Logging ☐ Backup ## Segurança ☐ Authentication ☐ Authorization ☐ Rate Limiting ☐ Input Validation ☐ Security Headers ☐ HTTPS ## Performance ☐ Cache ☐ Queue ☐ Database Indexes ☐ Pagination ☐ Lazy Loading

8. INSTRUÇÕES FINAIS O código gerado deve ser:  Funcional e Executável - Pronto para rodar  Bem Comentado - Comentários em português/inglês  Testável - Com suíte de testes  Documentado - Documentação completa  Preparado para IA - Com integração MCP A estrutura deve ser:  Modular - Fácil de navegar  Escalável - Pronta para crescer  Mantível - Fácil de modificar  Independente - Baixo acoplamento  Preparada para o Futuro - MCP integrado

9. ENTREGÁVEIS FINAIS FORNEÇA: Estrutura de pastas completa Código completo e funcional Testes automatizados Documentação completa Configurações Docker CI/CD pipelines Scripts de automação Exemplos de uso MCP LINGUAGEM: [PHP | Node.js | Python | Java | Go | Rust] FRAMEWORK: [Laravel | Express | FastAPI | Spring Boot | Gin | Axum] TIPO: [E-commerce | API | Microserviço | SaaS | Sistema Interno] ENTIDADE: [Product | User | Order | Customer | Document] FORNEÇA O PROJETO COMPLETO, PROFISSIONAL E PRONTO PARA PRODUÇÃO! text --- ## 2. **PROMPT POR LINGUAGEM** {#prompt-linguagem} ###  PHP/Laravel LINGUAGEM: PHP 8.2 FRAMEWORK: Laravel 10 TIPO: E-commerce ENTIDADE: Product TRANSPORTE MCP: streamable-http REQUISITOS TÉCNICOS: PHP 8.2 com typed properties e readonly classes Laravel 10 com Eloquent e Query Builder Laravel Sanctum para autenticação Laravel Cashier para pagamentos Laravel Horizon para filas Laravel Telescope para debug PHPUnit para testes PHPStan nível 9 para análise estática Laravel Pint para code style Spatie/Laravel-Permission para RBAC FastMCP para integração MCP ESTRUTURA ESPECÍFICA: app/Domain/ (Entities, ValueObjects, Events, Exceptions) app/Application/ (Commands, Queries, Handlers, DTOs) app/Infrastructure/ (Repositories,

Services, MCP) app/Http/Controllers/ (Presentation) app/Http/Requests/ (Validation) app/Http/Resources/ (Transformation) MCP IMPLEMENTAÇÃO: Usar FastMCP (<https://github.com/modelcontextprotocol/fastmcp>) Gerar ferramentas automaticamente a partir dos controllers Suporte a streamable-http para deploy na nuvem FORNEÇA CÓDIGO COMPLETO COM: Migrations Seeders Factories Tests Documentation Docker setup text ### 🟦 Node.js/TypeScript LINGUAGEM: TypeScript 5.0 FRAMEWORK: Express.js + TypeORM TIPO: API REST + MCP ENTIDADE: User TRANSPORTE MCP: stdio e streamable-http REQUISITOS TÉCNICOS: TypeScript strict mode Express.js com middleware TypeORM para banco de dados JWT para autenticação Winston para logs Bull + Redis para filas Jest para testes ESLint + Prettier @modelcontextprotocol/sdk Zod para validação de schemas ESTRUTURA ESPECÍFICA: src/domain/ (Entities, ValueObjects, Events, Repositories) src/application/ (Commands, Queries, Handlers, DTOs) src/infrastructure/ (Repositories, Services, MCP) src/presentation/ (Controllers, Middleware, Routes) MCP IMPLEMENTAÇÃO: Usar @modelcontextprotocol/sdk Servidor separado ou integrado (sidecar) Suporte a stdio (Claude Desktop) e streamable-http (deploy cloud) FORNEÇA CÓDIGO COMPLETO COM: TypeScript config Express config TypeORM entities JWT auth Tests Documentation Docker setup text ### 🟪 Python/FastAPI LINGUAGEM: Python 3.11 FRAMEWORK: FastAPI + SQLAlchemy TIPO: Microserviço + MCP ENTIDADE: Order TRANSPORTE MCP: streamable-http REQUISITOS TÉCNICOS: Python 3.11 com type hints FastAPI com OpenAPI/Swagger SQLAlchemy 2.0 para ORM Alembic para migrações Pydantic 2.0 para validação Pytest para testes Celery + Redis para filas Black + Ruff para code style FastMCP para integração MCP ESTRUTURA ESPECÍFICA: src/domain/ (entities, value_objects, events, repositories) src/application/ (commands, queries, handlers, dto) src/infrastructure/ (repositories, services, mcp) src/presentation/ (controllers, schemas, routers) MCP IMPLEMENTAÇÃO: Usar fastmcp (<https://github.com/jlowin/fastmcp>) Decorators para expor ferramentas Suporte a streamable-http FORNEÇA CÓDIGO COMPLETO COM: FastAPI app SQLAlchemy models Alembic migrations Pytest tests Documentation Docker setup text ### 🟡 Java/Spring Boot LINGUAGEM: Java 17 FRAMEWORK: Spring Boot 3.0 + JPA TIPO: Sistema Corporativo + MCP ENTIDADE: Customer TRANSPORTE MCP: streamable-http REQUISITOS TÉCNICOS: Java 17 com records e sealed classes Spring Boot 3.0 Spring Data JPA Spring Security com JWT Spring Cache (Redis) Spring Cloud Stream (RabbitMQ) JUnit 5 + Mockito Maven/Gradle Lombok Flyway para migrações Spring AI para integração MCP ESTRUTURA ESPECÍFICA: src/main/java/com/project/domain/ src/main/java/com/project/application/ src/main/java/com/project/infrastructure/ src/main/java/com/project/presentation/ MCP IMPLEMENTAÇÃO: Usar Spring AI (ou implementação customizada) Expor controllers com suporte a MCP Suporte a streamable-http FORNEÇA CÓDIGO COMPLETO COM: Spring Boot app JPA entities Spring Security config JUnit tests Documentation Docker setup text --- ## 3. **PROMPT POR TIPO DE PROJETO** {#prompt-projeto} ### 🛒 E-commerce TIPO: E-commerce Completo ENTIDADE PRINCIPAL: Product MÓDULOS ADICIONAIS: Order, Payment, User, Cart, Wishlist REQUISITOS FUNCIONAIS: Catálogo de produtos com categorias Carrinho de compras (session ou DB) Checkout com múltiplos métodos de pagamento Gestão de pedidos (status, histórico) Wishlist (lista de desejos) Avaliações e reviews Sistema de cupons e descontos Frete (cálculo, rastreamento) Notificações (email, SMS, push) Admin panel completo Relatórios e analytics SEO (metatags, sitemap) Multi-idioma MCP FERRAMENTAS: create_product list_products (com filtros) get_product update_product delete_product adjust_stock create_order get_order_status apply_coupon calculate_shipping add_to_wishlist FORNEÇA CÓDIGO COMPLETO PARA TODOS OS MÓDULOS text ### 🔔 API REST + MCP TIPO: API REST + MCP ENTIDADE PRINCIPAL: Resource REQUISITOS FUNCIONAIS: CRUD completo da entidade principal Filtros e paginação Busca full-text Autenticação JWT RBAC (Role-Based Access Control) Rate Limiting CORS configurado Logs estruturados (JSON) Monitoramento (Health checks, metrics) Documentação OpenAPI (Swagger) Versionamento de API MCP FERRAMENTAS: create_resource list_resources (com filtros) get_resource update_resource delete_resource search_resources get_resource_stats FORNEÇA CÓDIGO COMPLETO DA API E MCP text ### 🌐 Microserviço TIPO:

Microsserviço + MCP ENTIDADE PRINCIPAL: DomainEntity REQUISITOS FUNCIONAIS: Serviço independente Comunicação síncrona (REST/gRPC) Comunicação assíncrona (RabbitMQ/Kafka) Service Discovery (Consul/Eureka) Config Server API Gateway Circuit Breaker (Resilience4j) Distributed Tracing (Jaeger) Health Checks Metrics (Prometheus) Docker/Kubernetes MCP FERRAMENTAS: Todas as operações CRUD Métodos específicos do domínio FORNEÇA CÓDIGO COMPLETO DO MICROSERVIÇO text --- ## 4. **PROMPT PARA MÓDULOS MCP** {#prompt-modulo} BASEADO NA ARQUITETURA DEFINIDA, CRIE UM MÓDULO MCP COMPLETO PARA: DOMÍNIO: [Products/Orders/Users/Payments] ENTIDADE: [Product/Order/User/Payment]

1. SERVIDOR MCP Crie um servidor MCP completo que exponha: Ferramentas MCP (CRUD + Custom) create_[entity] list_[entities] (com paginação e filtros) get_[entity] update_[entity] delete_[entity] [CustomAction1] [CustomAction2] [CustomAction3] Recursos MCP [entity]:/api/[entities]/{id} [entities]:/api/[entities] (listagem paginada) [custom]:/api/[entities]/custom (se aplicável) Prompts MCP help_[entity] - Guia de uso create_[entity]_guide - Guia para criação troubleshoot_[entity] - Troubleshooting

2. IMPLEMENTAÇÃO Para CADA ferramenta, forneça: Schema (Zod/Pydantic/JsonSchema) Descrição clara para LLMs Handler (reutilizando Application Handlers) Formatação de resposta Tratamento de erros

3. TESTES Forneça testes para: Cada ferramenta Recursos Prompts Integração

4. DOCUMENTAÇÃO Forneça: Lista completa de ferramentas Exemplos de uso Exemplos de integração com Claude Desktop LINGUAGEM: [PHP/Node.js/Python/Java] FRAMEWORK: [Laravel/Express/FastAPI/Spring Boot] DOMÍNIO: [Products/Orders/Users] text --- ## 5. **PROMPT PARA REFATORAÇÃO** {#prompt-refatoracao} VOCÊ É UM ARQUITETO DE SOFTWARE ESPECIALISTA EM REFATORAÇÃO. PROJETO EXISTENTE: Linguagem: [LINGUAGEM] Framework: [FRAMEWORK] Estrutura Atual: [MVC | Monolítica | Outra] Funcionalidades: [Lista de funcionalidades] OBJETIVO: Refatorar o projeto existente para adotar: Arquitetura Hexagonal Separar em camadas (Domain, Application, Infrastructure, Presentation) Extrair interfaces de repositório Criar DTOs Extrair Handlers Padrões de Projeto Repository Pattern Service Pattern Command Pattern Factory Pattern Boas Práticas SOLID DRY TDD (escrever testes primeiro)  MCP Integration Criar servidor MCP Expor funcionalidades como ferramentas MCP Configurar para Claude Desktop PLANO DE REFATORAÇÃO Fase 1: Análise ☐ Mapear funcionalidades existentes ☐ Identificar violações de SOLID ☐ Mapear dependências Fase 2: Domain Layer ☐ Extrair entidades ☐ Criar value objects ☐ Definir interfaces de repositório Fase 3: Application Layer ☐ Criar Commands/Queries ☐ Criar Handlers ☐ Criar DTOs Fase 4: Infrastructure Layer ☐ Implementar repositórios ☐ Criar serviços externos ☐  Criar MCP Server Fase 5: Presentation Layer ☐ Refatorar controllers ☐ Adicionar requests com validação ☐ Adicionar resources Fase 6: Tests ☐ Escrever testes unitários ☐ Escrever testes de integração ☐  Escrever testes MCP Fase 7: Documentation ☐ Atualizar README ☐ Documentar MCP ☐ Criar ADRs ENTREGÁVEIS Código refatorado Novos testes Documentação atualizada MCP Server funcionando Guia de migração FORNEÇA O CÓDIGO COMPLETO E O PASSO A PASSO DA REFATORAÇÃO. text --- ## 6. **PROMPT PARA TESTES MCP** {#prompt-testes} CRIE UMA SUÍTE COMPLETA DE TESTES PARA O MÓDULO MCP: DOMÍNIO: [Products/Orders/Users] ENTIDADE: [Product/Order/User] LINGUAGEM: [PHP/Node.js/Python/Java]

1. TESTES DE FERRAMENTAS Para CADA ferramenta MCP, crie: Teste de Criação (Create) ☐ Teste com dados válidos ☐ Teste com dados inválidos (validação) ☐ Teste com autenticação (se aplicável) ☐ Teste de duplicata (se aplicável) ☐ Teste de Listagem (List) ☐ Teste de paginação ☐ Teste de filtros ☐ Teste de ordenação ☐ Teste de busca ☐ Teste de Obtenção (Get) ☐ Teste com ID válido ☐ Teste com ID inválido (404) ☐ Teste com permissões ☐ Teste de Atualização (Update) ☐ Teste com dados válidos ☐ Teste com dados inválidos ☐ Teste de concorrência ☐ Teste de Deleção (Delete) ☐ Teste com ID válido ☐ Teste com ID inválido ☐ Teste com permissões ☐ Teste Customizado ☐ [CustomAction1] - [descrição] ☐ [CustomAction2] - [descrição] ☐

2. TESTES DE RECURSOS ☐ Teste de recurso único ☐ Teste de coleção ☐ Teste de cache (se aplicável) ☐

3. TESTES DE PROMPTS ☐ Teste de prompt de ajuda ☐ Teste de prompt customizado ☐

4. TESTES DE INTEGRAÇÃO ☐ Teste de fluxo completo (create → get → update → delete) ☐ Teste com autenticação real ☐ Teste com banco real ☐

5. EXEMPLOS DE CÓDIGO Forneça exemplos de código para cada

tipo de teste. LINGUAGEM: [PHP/Node.js/Python/Java] FRAMEWORK DE TESTE: [PHPUnit/Pytest/Jest/JUnit]

text --- ## 7. ****PROMPT PARA DOCUMENTAÇÃO**** {#prompt-documentacao} CRIE DOCUMENTAÇÃO COMPLETA PARA O PROJETO COM MCP: PROJETO: [NOME_DO_PROJETO] LINGUAGEM: [PHP/Node.js/Python/Java] ENTIDADE: [Product/Order/User] 1. README.md markdown # 🚀 [NOME_DO_PROJETO] [Badges de status, coverage, etc] ## 📄 Índice [Índice completo] ## 📖 Sobre [Descrição do projeto] ## 🛠️ Tecnologias [Lista de tecnologias] ## 📋 Pré-requisitos [Lista de pré-requisitos] ## ⚙️ Instalação [Passos de instalação] ## ⚙️ Configuração [Passos de configuração] ## 🌐 Uso (REST) [Exemplos de API REST] ## 🤖 Uso (MCP) [Exemplos de uso com Claude Desktop] ## 🧪 Testes [Como rodar testes] ## 🚀 Deploy [Instruções de deploy] ## 🙌 Contribuição [Guia de contribuição] ## 📄 Licença [Informação da licença] 2. DOCUMENTAÇÃO MCP (docs/mcp/README.md) markdown # 🤖 MCP Integration - [NOME_DO_PROJETO] ## Sobre [Descrição da integração MCP] ## Configuração [Como configurar no Claude Desktop] ## Ferramentas Disponíveis [Lista completa de ferramentas com exemplos] ## Recursos Disponíveis [Lista completa de recursos] ## Prompts Disponíveis [Lista completa de prompts] ## Exemplos [Exemplos práticos de uso] ## Troubleshooting [Problemas comuns e soluções] 3. DOCUMENTAÇÃO DA API (docs/api/README.md) markdown # 🌐 API Documentation - [NOME_DO_PROJETO] ## Autenticação [Como autenticar] ## Endpoints [Lista de endpoints com exemplos] ## Schemas [Schemas de dados] ## Error Codes [Códigos de erro] 4. ARCHITECTURE DECISION RECORDS (docs/architecture/) ADR-001: Escolha da Arquitetura ADR-002: Decisão de usar MCP ADR-003: [Outras decisões importantes] 5. CONTRIBUTING.md markdown # 🙌 Guia de Contribuição ## Como contribuir [Passos para contribuir] ## Padrões de Código [Padrões a seguir] ## Processo de Review [Como funciona o review] ## Testes [Como escrever testes] FORNEÇA DOCUMENTAÇÃO COMPLETA E PRONTA PARA USO. text --- ## 8. ****PROMPT PARA DEVOPS COM MCP**** {#prompt-devops} CRIE CONFIGURAÇÕES DE DEVOPS COMPLETAS COM SUPORTE A MCP: PROJETO: [NOME_DO_PROJETO] LINGUAGEM: [PHP/Node.js/Python/Java] 1. DOCKER Dockerfile (Aplicação Principal) dockerfile FROM [base-image] WORKDIR /app # Install dependencies COPY package*.json ./ RUN npm install # Copy source COPY . . # Build RUN npm run build # Run CMD ["node", "dist/server.js"] Dockerfile (MCP Server) dockerfile FROM [base-image] WORKDIR /app # Install dependencies COPY package*.json ./ RUN npm install # Copy source COPY . . # Build MCP server RUN npm run build:mcp # Run MCP server CMD ["node", "dist/mcp/server.js"] docker-compose.yml yaml version: '3.8' services: app: # Configuração da aplicação mcp-server: # Configuração do MCP Server mcp-gateway: # Configuração do MCP Gateway db: # Configuração do banco redis: # Configuração do Redis nginx: # Configuração do Nginx (se aplicável) 2. CI/CD (GitHub Actions) .github/workflows/ci.yml yaml name: CI/CD Pipeline on: push: branches: [main, develop] pull_request: branches: [main] jobs: test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v3 - name: Setup - name: Install - name: Unit Tests - name: Integration Tests - name: MCP Tests build: needs: test runs-on: ubuntu-latest steps: - name: Build Docker - name: Push to Registry deploy: needs: build runs-on: ubuntu-latest if: github.ref == 'refs/heads/main' steps: - name: Deploy to Production 3. SCRIPTS DE AUTOMAÇÃO Makefile makefile .PHONY: help install test dev build docker-up docker-down help: @echo "Comandos disponíveis:" @echo " install Instalar dependências" @echo " test Rodar testes" @echo " dev Rodar em desenvolvimento" @echo " build Build para produção" @echo " docker-up Iniciar containers" @echo " docker-down Parar containers" install: [comando para instalar dependências] test: [comando para rodar testes] dev: [comando para rodar em desenvolvimento] build: [comando para build] docker-up: docker-compose up -d docker-down: docker-compose down 4. MONITORAMENTO Prometheus metrics Health checks Logs estruturados (JSON) Distributed tracing FORNEÇA CONFIGURAÇÕES COMPLETAS E PRONTAS PARA PRODUÇÃO. text --- ## 🎯 ****COMO USAR ESTES PROMPTS**** #### Passo 1: Escolha o Prompt Adequado - ****Projeto Novo****: Use o Prompt Base Completo - ****Linguagem Específica****: Use o Prompt por Linguagem - ****Tipo Específico****: Use o Prompt por Tipo - ****Módulo MCP****: Use o Prompt para Módulos - ****Refatoração****: Use o Prompt para Refatoração #### Passo

2: Substitua os Placeholders ``yaml [NOME_DO_PROJETO]: MeuEcommerce [LINGUAGEM]: PHP 8.2 [FRAMEWORK]: Laravel 10 [TIPO]: E-commerce [ENTIDADE]: Product [DOMÍNIO]: Products Passo 3: Execute no Assistente IA Cole o prompt completo Aguarde o código ser gerado Revise e adapte Passo 4: Valide o Resultado ☐ Estrutura de pastas criada ☐ Código funcional ☐ MCP Server configurado ☐ Testes implementados ☐ Documentação completa ☐ Docker configurado ☐ CI/CD configurado 📖 TABELA DE REFERÊNCIA RÁPIDA Linguagem Framework MCP SDK Testes PHP Laravel FastMCP PHPUnit/Pest Node.js Express @modelcontextprotocol/sdk Jest Python FastAPI fastmcp Pytest Java Spring Boot Spring AI JUnit Go Gin mcp-go testing Rust Axum mcp-rs cargo test 🚀 EXEMPLO PRONTO PARA USAR text VOCÊ É UM ARQUITETO DE SOFTWARE ESPECIALISTA EM PROJETOS COM MCP. CRIE UM PROJETO COMPLETO DE E-COMMERCE COM: LINGUAGEM: PHP 8.2 FRAMEWORK: Laravel 10 TIPO: E-commerce ENTIDADE: Product REQUISITOS: - CRUD de produtos com categorias - Autenticação JWT (Laravel Sanctum) - Permissões (Spatie/Laravel-Permission) - Testes com PHPUnit - Documentação com Swagger - MCP com FastMCP - Docker - CI/CD (GitHub Actions) FORNEÇA CÓDIGO COMPLETO E FUNCIONAL. Use estes prompts para criar projetos profissionais com MCP em qualquer linguagem! 🚀